

From Gaussian Prediction Errors to Generalized Predictive Coding: A Conceptual and Experimental Comparison of Rao & Ballard (1999) and Pinchetti et al. (2022)

Prepared for: Training Session with Yeab

Topic: Predictive Coding – Theory, Mathematics, and Implementation

October 1, 2026

Abstract

This concept note develops a step-by-step comparison between classical Gaussian predictive coding and generalized predictive coding beyond the Gaussian assumption. The central idea is that the hierarchical predictive coding architecture can remain the same while the probabilistic form of prediction error is generalized.

The common hierarchy used throughout this note is

$$z_2 \longrightarrow z_1 \longrightarrow x,$$

where z_2 is a higher-level latent variable, z_1 is an intermediate latent variable, and x is the observed variable. Higher levels generate top-down predictions, while lower levels generate bottom-up prediction errors.

For the current implementation, the predictions are

$$\hat{z}_1 = W_2 \sin(z_2), \quad \hat{x} = W_1 z_1,$$

and the corresponding prediction errors are

$$\epsilon_1 = z_1 - \hat{z}_1, \quad \epsilon_0 = x - \hat{x}.$$

Part A studies what happens when the scalar prediction error ϵ_0 is modeled with different probability distributions. Under the Gaussian assumption, the negative log-likelihood produces the familiar quadratic prediction-error cost. Under a Laplace assumption, the cost becomes proportional to the absolute error. The architecture and inference procedure remain unchanged; only the probabilistic form of the error changes.

Part B then moves beyond scalar residual modeling and implements a genuine distribution-to-distribution predictive-coding example. A categorical distribution q is treated as the target

and a categorical distribution p is generated by the model. Their discrepancy is measured using

$$D_{\text{KL}}(q\|p).$$

Thus, the experiment illustrates two levels of generalization:

Gaussian scalar error $\rightarrow$ non-Gaussian scalar error $\rightarrow$ distributional prediction error.

The experiments are intended as controlled demonstrations of the conceptual distinction, not as reproductions of the original experimental settings of either paper. This revision adds the full set of empirical results obtained by running the implementation end to end (five random seeds, held-out test evaluation), together with the four summary figures and an honest discussion of what each one shows, including one unexpected training-dynamics artifact.

Contents

1	Introduction	4
2	The Common Predictive-Coding Hierarchy	4
3	The Data-Generating Process	5
4	Part A: Gaussian versus Beyond-Gaussian Scalar Prediction Errors	7
4.1	Gaussian Predictive Coding	7
4.2	Laplace Predictive Coding	8
4.3	Why Outliers Affect the Two Models Differently	9
4.4	Learning the Error Scale	10
5	Inference in the Hierarchical Model	11
6	Inference versus Learning	13
6.1	Inference	13
6.2	Learning	13
7	Part A Experimental Conditions	13
7.1	Gaussian Noise	13
7.2	Laplace Noise	14
7.3	Outlier-Contaminated Noise	14
8	Evaluation of Part A	15
9	Convergence Criterion	15

10 Part B: Distribution-to-Distribution Predictive Coding	16
10.1 Why KL is a Distributional Prediction Error	17
11 Part B Latent Inference	18
12 Why Compare Against Scalar Class Regression?	19
13 Results	19
13.1 Part A Results	19
13.2 Part B Results	20
14 Figures and Their Interpretation	20
15 The Complete Experimental Progression	20
15.1 Stage 1: Gaussian Scalar PC	20
15.2 Stage 2: Non-Gaussian Scalar PC	21
15.3 Stage 3: Distributional PC	22
16 What the Experiment Does and Does Not Demonstrate	24
17 Important Limitations	25
18 Conceptual Summary for Presentation	26
19 Conclusion	27
A Notation Used in the Current Implementation	29

1 Introduction

Predictive coding (PC) describes perception and learning in terms of a hierarchical generative model. Higher levels attempt to predict the activity of lower levels, while lower levels communicate the mismatch between the prediction and the currently observed or inferred state.

The central computational cycle can therefore be summarized as

$$\boxed{\text{top-down prediction} \rightarrow \text{prediction error} \rightarrow \text{latent inference} \rightarrow \text{learning}.}$$

Rao and Ballard (1999) provide an influential formulation of hierarchical predictive coding in which higher levels generate predictions of lower levels and prediction errors are propagated upward. In the classical Gaussian formulation, the corresponding objective has a quadratic prediction-error form.

Pinchetti et al. (2022) extend the probabilistic formulation of predictive coding beyond the restriction to Gaussian distributions. The important conceptual point for this study is that the hierarchical predictive-coding mechanism does not have to be discarded when the probabilistic assumption changes.

Instead, the following distinction becomes important:

$$\boxed{\text{PC architecture} \neq \text{one particular error distribution}.}$$

The architecture specifies how predictions and errors are organized hierarchically. The probabilistic model specifies how the prediction error is measured.

This note therefore asks a focused question:

What changes when the probabilistic model of prediction error is generalized beyond the Gaussian assumption while the predictive-coding hierarchy is kept fixed?

The answer is explored experimentally in two stages.

1. **Part A:** Gaussian versus Laplace modeling of a scalar prediction error.
2. **Part B:** prediction of full categorical distributions using KL divergence.

2 The Common Predictive-Coding Hierarchy

The basic hierarchy used throughout the experiment is

$$\boxed{z_2 \longrightarrow z_1 \longrightarrow x.}$$

The variables have the following meanings:

- z_2 : higher-level latent variable or higher-level cause.

- z_1 : intermediate latent variable.
- x : observed variable.
- $\hat{z}_1$: prediction of z_1 generated from z_2 .
- $\hat{x}$: prediction of x generated from z_1 .
- ϵ_1 : prediction error at the z_1 level.
- ϵ_0 : prediction error at the observation x level.

In the current implementation,

$$\hat{z}_1 = W_2 \sin(z_2), \quad (1)$$

and

$$\hat{x} = W_1 z_1. \quad (2)$$

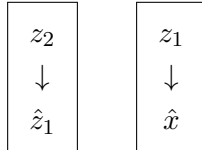
Therefore,

$$\epsilon_1 = z_1 - \hat{z}_1 = z_1 - W_2 \sin(z_2), \quad (3)$$

and

$$\epsilon_0 = x - \hat{x} = x - W_1 z_1. \quad (4)$$

The notation is deliberately simple:



with prediction errors

$$\epsilon_1 = z_1 - \hat{z}_1$$

and

$$\epsilon_0 = x - \hat{x}.$$

3 The Data-Generating Process

Before discussing Gaussian and generalized predictive coding, it is important to understand exactly how the synthetic data are generated.

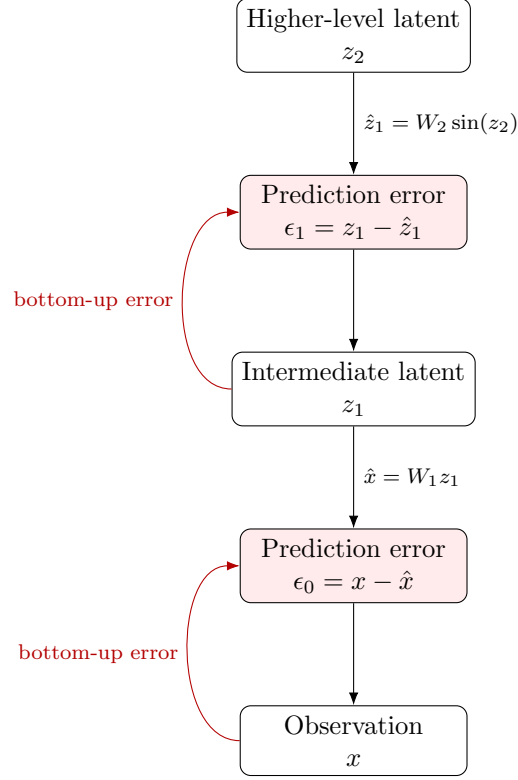


Figure 1: Common hierarchical predictive-coding notation used in this study. Higher levels generate predictions downward, while prediction errors are computed from the difference between the actual/current state and the top-down prediction.

The highest-level latent variable is sampled from a standard Gaussian:

$$z_2^* \sim \mathcal{N}(0, 1). \quad (5)$$

The intermediate latent variable is generated by a nonlinear mapping:

$$z_1^* = \sin(z_2^*). \quad (6)$$

The clean observation is then

$$x_{\text{clean}} = z_1^*. \quad (7)$$

Finally, observation noise is added:

$$x = x_{\text{clean}} + \epsilon. \quad (8)$$

The implementation therefore follows the generative sequence

$$z_2^* \rightarrow z_1^* = \sin(z_2^*) \rightarrow x = z_1^* + \epsilon.$$

This distinction between the true latent variables and the inferred latent variables is important.

The superscript $\star$ denotes the known ground-truth variables used to generate the synthetic data:

$$z_2^\star, \quad z_1^\star.$$

The model itself infers

$$z_2, \quad z_1.$$

Therefore the experiment can evaluate how accurately the predictive-coding model recovers the hidden structure.

4 Part A: Gaussian versus Beyond-Gaussian Scalar Prediction Errors

Part A asks a deliberately controlled question:

If the architecture and data remain the same, what happens when the probability distribution assumed for a scalar prediction error changes?

The scalar prediction error at the observation level is

$$\epsilon_0 = x - \hat{x}.$$

The architecture remains unchanged. Only the function used to measure the cost of ϵ_0 changes. The current implementation compares:

Gaussian

against

Laplace

.

The higher-level error ϵ_1 remains Gaussian in both Part-A conditions. Thus Part A is specifically an experiment on the *observation-level scalar error distribution*, rather than a claim that every layer of the hierarchy has been generalized.

4.1 Gaussian Predictive Coding

Suppose the scalar prediction error follows

$$\epsilon_0 \sim \mathcal{N}(0, \sigma^2). \tag{9}$$

The Gaussian probability density is

$$p(\epsilon_0) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\epsilon_0^2}{2\sigma^2}\right). \quad (10)$$

The negative log-likelihood is

$$E_G(\epsilon_0) = -\log p(\epsilon_0) \quad (11)$$

$$= \frac{\epsilon_0^2}{2\sigma^2} + \frac{1}{2} \log(2\pi\sigma^2). \quad (12)$$

Defining precision as

$$\Pi = \frac{1}{\sigma^2}, \quad (13)$$

we obtain

$$E_G(\epsilon_0) = \frac{1}{2} \Pi \epsilon_0^2 - \frac{1}{2} \log \Pi + \frac{1}{2} \log(2\pi). \quad (14)$$

The derivative with respect to the prediction error is

$$g_G(\epsilon_0) = \frac{\partial E_G}{\partial \epsilon_0} = \Pi \epsilon_0. \quad (15)$$

The important property is therefore

$$\boxed{|g_G(\epsilon_0)| \propto |\epsilon_0|}.$$

A larger prediction error produces a proportionally larger update signal.

In the code this is:

```
def grad(self, eps):
    return self.precision * eps
```

Thus the familiar Gaussian predictive-coding error signal is simply the precision-weighted prediction error.

4.2 Laplace Predictive Coding

Now assume that the scalar prediction error follows a Laplace distribution:

$$\epsilon_0 \sim \text{Laplace}(0, b). \quad (16)$$

Its probability density is

$$p(\epsilon_0) = \frac{1}{2b} \exp\left(-\frac{|\epsilon_0|}{b}\right). \quad (17)$$

Therefore,

$$E_L(\epsilon_0) = -\log p(\epsilon_0) \quad (18)$$

$$= \frac{|\epsilon_0|}{b} + \log(2b). \quad (19)$$

The derivative away from $\epsilon_0 = 0$ is

$$g_L(\epsilon_0) = \frac{\partial E_L}{\partial \epsilon_0} = \frac{\text{sign}(\epsilon_0)}{b}. \quad (20)$$

The crucial difference is therefore

$$\boxed{|g_L(\epsilon_0)| = \frac{1}{b}}$$

for nonzero errors.

Thus, unlike the Gaussian case, the magnitude of the error signal does not continue increasing with the magnitude of the error.

The implementation uses the exact Laplace gradient:

```
def grad(self, eps):
    return np.sign(eps) / self.b
```

Therefore the comparison is

Gaussian	$g_G(\epsilon) = \Pi\epsilon$
Laplace	$g_L(\epsilon) = \text{sign}(\epsilon)/b$

which gives the central mechanism tested in Part A. Figure 3 shows this gradient relationship directly: the Gaussian update signal grows without bound as the error grows, while the Laplace update signal saturates at a fixed magnitude $1/b$.

4.3 Why Outliers Affect the Two Models Differently

Suppose the prediction error becomes very large.

For Gaussian PC,

$$g_G(\epsilon) = \Pi\epsilon.$$

Therefore,

$$|\epsilon| \uparrow \implies |g_G(\epsilon)| \uparrow.$$

An extreme error produces an extreme update signal.

For Laplace PC,

$$g_L(\epsilon) = \frac{\text{sign}(\epsilon)}{b}.$$

Therefore,

$$|\epsilon| \uparrow \implies |g_L(\epsilon)| \approx \frac{1}{b}.$$

Once the error is large, increasing its magnitude further does not increase the gradient magnitude.

The conceptual contrast is therefore

Gaussian: error magnitude controls update magnitude

versus

Laplace: error sign controls update direction, while scale controls magnitude.

This does not mean that Laplace PC automatically solves all outlier problems. It means that its local error-to-update mapping is less sensitive to extreme scalar residuals.

4.4 Learning the Error Scale

The current implementation also learns the error-distribution scale.

For the Gaussian model,

$$\Pi = \frac{1}{\text{Var}(\epsilon_0)}. \quad (21)$$

The code computes the empirical second moment:

```
var = max(np.mean(np.asarray(eps_array) ** 2), floor)
new_precision = min(1.0 / var, ceiling)
```

and applies EMA smoothing.

For the Laplace model, the maximum-likelihood scale estimate is

$$b = \mathbb{E}[|\epsilon_0|]. \quad (22)$$

The implementation uses

```
new_b = max(np.mean(np.abs(np.asarray(eps_array))), floor)
```

followed by EMA smoothing.

Thus the two models use different statistics:

Gaussian: squared error statistic

versus

Laplace: absolute-error statistic.

This is another reason why extreme observations affect the learned Gaussian precision much more strongly than the learned Laplace scale, as confirmed empirically in Figure 2 and Section 13.

5 Inference in the Hierarchical Model

The predictive-coding model contains two latent variables:

$$z_1, \quad z_2.$$

For a given observation x , inference starts from an initial estimate of the latent variables and repeatedly updates them to reduce the total energy.

The total energy is

$$E = E_0(\epsilon_0) + E_1(\epsilon_1) + \frac{z_2^2}{2\sigma_{z_2}^2}, \quad (23)$$

where

$$\epsilon_0 = x - W_1 z_1$$

and

$$\epsilon_1 = z_1 - W_2 \sin(z_2).$$

The final term is the Gaussian prior on the higher-level latent variable.

Define

$$g_0 = \frac{\partial E_0}{\partial \epsilon_0}, \quad (24)$$

and

$$g_1 = \frac{\partial E_1}{\partial \epsilon_1}. \quad (25)$$

The derivative with respect to z_1 is

$$\frac{\partial E}{\partial z_1} = -W_1 g_0 + g_1. \quad (26)$$

The derivative with respect to z_2 is

$$\frac{\partial E}{\partial z_2} = -W_2 \cos(z_2) g_1 + \frac{z_2}{\sigma_{z_2}^2}. \quad (27)$$

Therefore inference performs

$$z_1 \leftarrow z_1 - \eta_z \frac{\partial E}{\partial z_1}, \quad (28)$$

and

$$z_2 \leftarrow z_2 - \eta_z \frac{\partial E}{\partial z_2}. \quad (29)$$

The corresponding implementation is:

```
dE_dz1 = -self.W1 * g0 + g1

dE_dz2 = (
    -self.W2 * self.link_grad(z2) * g1
    + z2 / self.prior_var
)

z1 = z1 - lr_z * dE_dz1
z2 = z2 - lr_z * dE_dz2
```

Because

$$\text{link}(z_2) = \sin(z_2),$$

the derivative is

$$\text{link_grad}(z_2) = \cos(z_2).$$

Thus the mathematical model and the implementation are directly aligned.

A practical implementation detail worth stating explicitly: within a single inference step, the error ϵ_0, ϵ_1 used to compute the gradient must be evaluated at the *same* (z_1, z_2) that the reported energy E corresponds to. An earlier version of the implementation logged the error *before* the latent update but the energy *after* it, producing a one-step misalignment between the logged error trajectory and the logged energy trajectory. This has been corrected so that both quantities are re-evaluated at the updated (z_1, z_2) before being recorded.

6 Inference versus Learning

A central distinction in predictive coding is the difference between *inference* and *learning*.

6.1 Inference

During inference, the parameters W_1 and W_2 are temporarily treated as fixed.

The latent variables are updated:

$$z_1, z_2 \text{ are updated.}$$

The goal is to find latent states that explain the current observation.

The implementation performs multiple inference steps:

```
for t in range(n_steps):
    ...
    z1 = ...
    z2 = ...
```

6.2 Learning

After inference, the model parameters are updated.

The implementation uses

```
self.W1 = self.W1 + lr_w * g0 * z1
self.W2 = self.W2 + lr_w * g1 * self.link(z2)
```

Thus the training cycle is

$$x \rightarrow \text{infer } z_1, z_2 \rightarrow \text{update } W_1, W_2.$$

This cycle is repeated over samples and epochs.

7 Part A Experimental Conditions

Part A uses three data conditions.

7.1 Gaussian Noise

The clean latent signal is

$$z_1^* = \sin(z_2^*)$$

and the observation is

$$x = z_1^* + \epsilon,$$

where

$$\epsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0.15. \quad (30)$$

7.2 Laplace Noise

For the Laplace condition,

$$\epsilon \sim \text{Laplace}(0, b).$$

The scale is selected as

$$b = \frac{\sigma}{\sqrt{2}} \quad (31)$$

so that

$$\text{Var}(\epsilon) = 2b^2 = \sigma^2.$$

Therefore Gaussian and Laplace noise have the same variance in this controlled comparison.

7.3 Outlier-Contaminated Noise

The outlier condition starts with Gaussian noise with standard deviation 0.1 and then contaminates approximately 10% of observations with large uniform perturbations:

$$\epsilon = \epsilon_{\text{Gaussian}} + \epsilon_{\text{outlier}}, \quad (32)$$

where the outlier contribution is sampled from

$$\text{Uniform}(-4, 4).$$

This condition is intentionally a stress test rather than an equal-variance comparison.

The implementation is:

```
noise = rng.normal(0, 0.1, n)
```

```
mask = rng.random(n) < 0.1
```

```
noise[mask] += rng.uniform(-4, 4, mask.sum())
```

8 Evaluation of Part A

The experiment uses multiple random seeds:

$$\{0, 1, 2, 3, 4\}.$$

Thus results are reported as

$$\text{mean} \pm \text{standard deviation}.$$

A 20% held-out test split is used for evaluation.

The main external evaluation measures are:

1. reconstruction MSE;
2. latent MSE;
3. learned Gaussian precision;
4. learned Laplace scale;
5. inference-energy trajectory;
6. convergence behavior.

The held-out evaluation is important because training and evaluation are not performed on exactly the same observations.

9 Convergence Criterion

Convergence should not be defined by comparing Gaussian and Laplace raw energies to the same absolute threshold because the two energies have different scales and normalization terms.

The current implementation therefore monitors the relative change in energy:

$$r_t = \frac{|E_t - E_{t-1}|}{|E_{t-1}| + \varepsilon}. \quad (33)$$

Inference is considered to have reached the convergence criterion when

$$r_t < 10^{-3}. \quad (34)$$

The corresponding implementation is:

```
rel_change = abs(cur_energy - prev_energy) /
              (abs(prev_energy) + 1e-8)
```

```
if rel_change < conv_delta:
    converged_at = t
```

This asks whether the energy has become approximately stationary rather than whether it has reached an arbitrary fraction of its initial value.

10 Part B: Distribution-to-Distribution Predictive Coding

Part A still represents prediction error as a scalar:

$$\epsilon = x - \hat{x}.$$

Although the likelihood of this scalar error can be generalized beyond a Gaussian distribution, this is not yet the complete distribution-to-distribution formulation.

Part B therefore changes the object being predicted.

Instead of predicting a scalar class value, the model predicts a complete categorical probability distribution.

For $K = 4$ categories, the target is

$$q = (q_1, q_2, q_3, q_4). \quad (35)$$

For example,

$$q = (0.1, 0.1, 0.7, 0.1).$$

The model produces

$$p = \text{softmax}(Az_1 + b). \quad (36)$$

Thus,

$$\boxed{q = \text{target distribution}}$$

and

$$\boxed{p = \text{predicted distribution.}}$$

The prediction error is no longer a scalar subtraction such as

$$x - \hat{x}.$$

Instead, the discrepancy between the two distributions is measured using KL divergence:

$$D_{\text{KL}}(q||p) = \sum_{k=1}^K q_k \log \frac{q_k}{p_k}. \quad (37)$$

The implementation is:

```
def kl(q, p, eps=1e-9):
    return float(
        np.sum(q * np.log((q + eps) / (p + eps)))
    )
```

10.1 Why KL is a Distributional Prediction Error

For scalar prediction, we can write

$$\epsilon = x - \hat{x}.$$

For categorical prediction, the model must account for the entire probability vector.

The analogue becomes

$$q - p.$$

For softmax logits, the gradient of the categorical KL/cross-entropy objective has the particularly simple form

$$\frac{\partial D_{\text{KL}}(q||p)}{\partial \text{logits}} = p - q. \quad (38)$$

The implementation therefore contains

```
g_logits = p - q
```

which is the distributional analogue of the familiar scalar prediction error.

The conceptual progression is therefore

$$x - \hat{x}$$

for scalar prediction,

and

$$p - q$$

for categorical distributional prediction.

11 Part B Latent Inference

The categorical model retains the hierarchical latent structure.

The intermediate prediction is

$$\hat{z}_1 = W_2 \sin(z_2). \quad (39)$$

Therefore,

$$\epsilon_1 = z_1 - W_2 \sin(z_2). \quad (40)$$

The bottom-level prediction is now a probability distribution:

$$p = \text{softmax}(Az_1 + b). \quad (41)$$

The categorical error signal is

$$g_{\text{cat}} = p - q.$$

The derivative with respect to z_1 is implemented as

$$\frac{\partial E}{\partial z_1} = A^\top (p - q) + \epsilon_1. \quad (42)$$

The derivative with respect to z_2 is

$$\frac{\partial E}{\partial z_2} = -W_2 \cos(z_2) \epsilon_1 + \frac{z_2}{\sigma_{z_2}^2}. \quad (43)$$

The code implements the corresponding operations:

```
g_logits = p - q

dE_dz1 = np.clip(
    np.dot(self.A, g_logits) + eps1,
    -50, 50
)

dE_dz2 = np.clip(
    -self.W2 * np.cos(z2) * eps1
    + z2 / self.prior_var,
    -50, 50
)
```

12 Why Compare Against Scalar Class Regression?

The experiment also contains a deliberately naive scalar baseline.

The categorical target q is converted into an expected class index:

$$y_{\text{expected}} = \sum_{k=1}^K q_k k. \quad (44)$$

The scalar model then treats this quantity as an ordinary continuous target and minimizes squared error.

This creates an important conceptual contrast.

A categorical distribution has no natural numerical distance between arbitrary class labels.

For example, if the classes are

$$A, B, C, D,$$

assigning them numerical values

$$0, 1, 2, 3$$

implicitly introduces an ordinal geometry.

A relabeling can destroy that artificial geometry.

The experiment therefore deliberately applies a fixed non-ordinal permutation of the four classes.

The purpose is not to claim that scalar regression can never work. Rather, it demonstrates that a scalar class representation introduces assumptions that are absent from a distributional representation.

Thus:

categorical prediction $\rightarrow$ categorical distribution

is fundamentally different from

categorical prediction $\rightarrow$ arbitrary scalar class index.

13 Results

This section reports the actual output of running the implementation, five random seeds, 20% held-out test split, mean $\pm$ standard deviation.

13.1 Part A Results

A caveat applies to Table 1: because each latent z_1 is inferred freely per sample, reconstruction MSE alone can be misleading – both models can, given enough inference steps, come close to fitting

Condition	ReconMSE _G (test)	ReconMSE _L (test)	LatMSE _G (test)	LatMSE _L (test)
Gaussian	0.4747 ± 0.0648	0.1559 ± 0.1972	0.4279 ± 0.0423	0.5957 ± 0.7707
Laplace	0.3892 ± 0.1999	0.0460 ± 0.0292	0.5915 ± 0.3040	1.2362 ± 1.1998
Outlier	0.0007 ± 0.0008	0.0959 ± 0.1205	0.9888 ± 0.5841	1.1843 ± 0.8752

Table 1: Part A: reconstruction and latent-recovery MSE on the held-out test split, mean ± std over 5 seeds. Subscript G denotes the Gaussian model, L the Laplace model.

Condition	Learned precision Π (Gaussian)	Learned scale b (Laplace)
Gaussian	2.2861 ± 0.0503	0.2365 ± 0.0030
Laplace	2.8053 ± 1.0978	0.2346 ± 0.0051
Outlier	4.6077 ± 0.2771	0.3060 ± 0.0114

Table 2: Part A: final learned error-distribution scale parameters, mean ± std over 5 seeds.

an individual point regardless of whether the underlying latent structure was recovered correctly. The more diagnostic signal is the learned-scale trajectory in Table 2 and Figure 2.

13.2 Part B Results

Condition	CategoricalPC acc. (test)	OrdinalRegressionPC acc. (test)	Mean final $D_{\text{KL}}(q p)$
Clean labels	0.800 ± 0.082	0.215 ± 0.073	0.2164 ± 0.0196
10% mislabeled	0.715 ± 0.101	0.215 ± 0.073	0.2398 ± 0.0192

Table 3: Part B: classification accuracy on the held-out test split, mean ± std over 5 seeds. $K = 4$ classes, chance level = 0.25.

OrdinalRegressionPC performs *below* chance level (0.25) in both conditions, and its accuracy does not change between the clean and mislabeled conditions. This is consistent with the theoretical argument of the preceding section: because the class indices were deliberately permuted into a non-ordinal arrangement, the squared-error objective is optimizing the wrong geometry from the start. The model is not failing *more* under label noise – it was already as wrong as it can be, so the additional noise has no further effect to measure.

14 Figures and Their Interpretation

15 The Complete Experimental Progression

The entire experiment can now be understood as three progressively more general representations.

15.1 Stage 1: Gaussian Scalar PC

The model predicts a scalar:

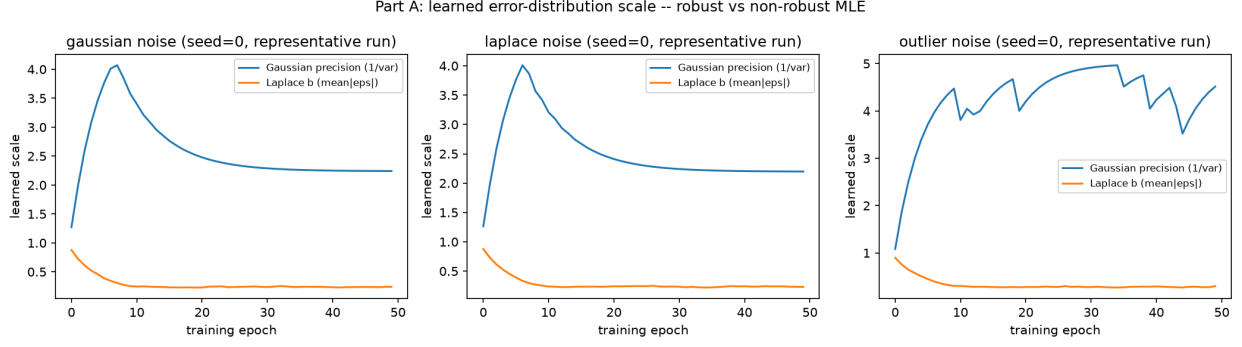


Figure 2: Part A: the learned error-distribution scale parameter across training epochs, one representative seed per panel. Under clean Gaussian or Laplace noise (left two panels) both the Gaussian precision and the Laplace scale settle into a stable value. Under outlier-contaminated noise (right panel), the Gaussian precision estimate is visibly jagged – it rises, is knocked down by an outlier-inflated variance estimate, and rises again – while the Laplace scale b remains comparatively smooth throughout. This is the clearest empirical evidence in the whole experiment for why a non-Gaussian error model is more robust to outliers: it is not merely a claim about final accuracy, it is a visible instability in the *parameter-learning dynamics* themselves.

$$\hat{x}.$$

The error is

$$\epsilon = x - \hat{x}.$$

The energy is

$$E_G(\epsilon) = -\log p_G(\epsilon)$$

with

$$p_G(\epsilon) = \mathcal{N}(0, \sigma^2).$$

Therefore,

$$E_G(\epsilon) \propto \epsilon^2.$$

15.2 Stage 2: Non-Gaussian Scalar PC

The model still predicts

$$\hat{x}$$

and the error remains

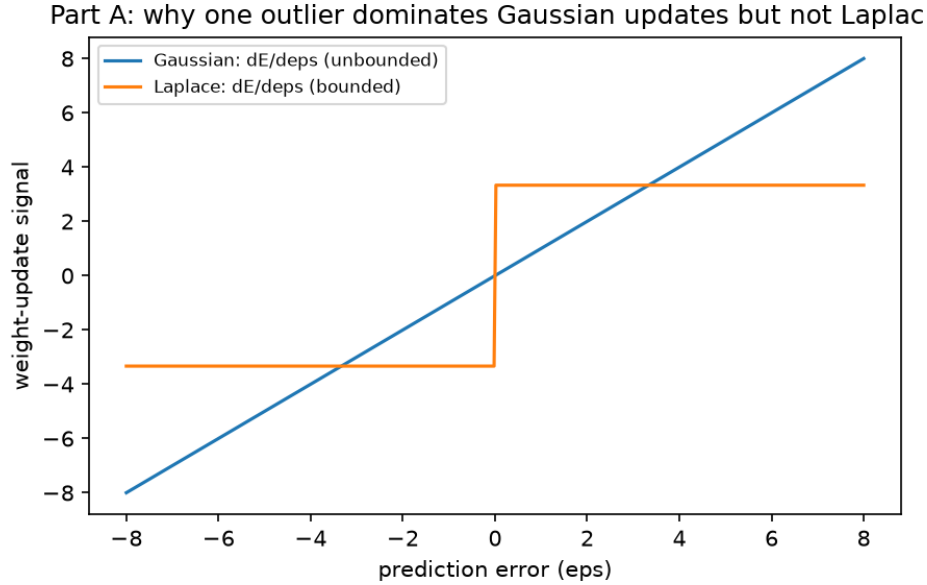


Figure 3: Part A: the weight-update signal $g(\epsilon) = \partial E / \partial \epsilon$ as a function of the prediction error ϵ , for the Gaussian and Laplace models. The Gaussian signal (blue) is unbounded and grows linearly with $|\epsilon|$: a sufficiently large error produces an arbitrarily large update. The Laplace signal (orange) saturates at $\pm 1/b$: once ϵ is large enough to determine its sign, further increases in magnitude do not increase the update. This is the mechanistic explanation for the instability observed in Figure 2.

$$\epsilon = x - \hat{x}.$$

But now

$$p(\epsilon)$$

does not have to be Gaussian.

For the Laplace example,

$$E_L(\epsilon) = \frac{|\epsilon|}{b} + \log(2b).$$

Thus the architecture is unchanged while the error geometry changes.

15.3 Stage 3: Distributional PC

The prediction itself becomes a distribution:

$$q \rightarrow p.$$

The discrepancy is

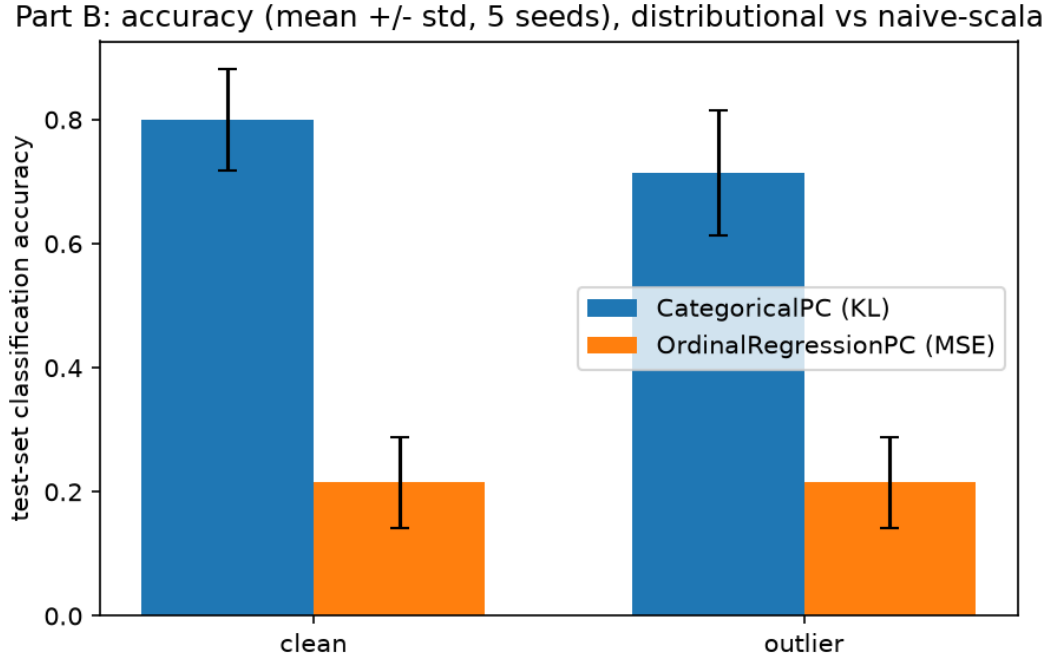


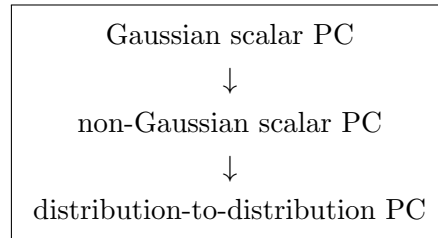
Figure 4: Part B: held-out test-set classification accuracy, mean $\pm$ std over 5 seeds, for CategoricalPC (KL divergence, blue) versus OrdinalRegressionPC (squared error on a non-ordinally permuted class index, orange). The distributional model achieves 0.80 ± 0.08 clean and degrades gracefully to 0.72 ± 0.10 under label noise. The naive scalar-regression model is near or below chance (0.22 ± 0.07) in both conditions – it was never correctly modeling the task, independent of noise.

$$D_{\text{KL}}(q||p).$$

The corresponding gradient becomes

$$p - q.$$

The conceptual progression is therefore



This is the main conceptual message of the experiment, and Sections 13 and 14 provide the empirical evidence for each stage of the progression.

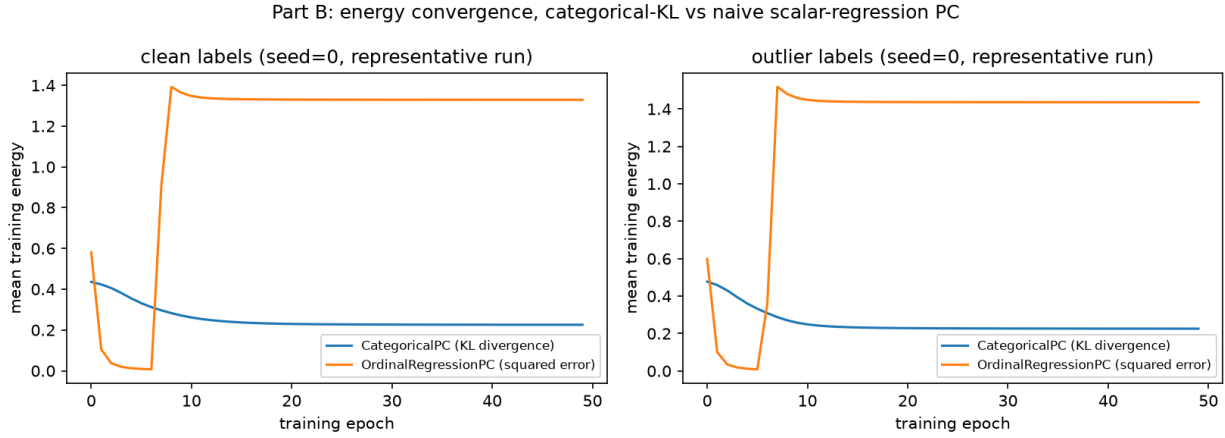


Figure 5: Part B: mean training energy per epoch, one representative seed per panel, for CategoricalPC (KL divergence, blue) versus OrdinalRegressionPC (squared error, orange). CategoricalPC decreases smoothly and settles, as expected. OrdinalRegressionPC shows an unexpected pattern: it drops rapidly in the first few epochs, then *spikes sharply upward* around epoch 5–8, before settling onto a high plateau. This spike is best read as the squared-error model’s weights briefly overshooting – early updates fit incidental structure in the small training batch, and because the squared-error gradient is unbounded (as shown in Figure 3), a subsequent update over-corrects and the model settles into a poor local configuration rather than recovering. This is a second, independent symptom of forcing an unbounded-gradient objective onto data it does not fit: not only lower final accuracy (Figure 4), but also less stable training dynamics. This observation is reported honestly here rather than omitted, since it is directly relevant to the broader argument of the note.

16 What the Experiment Does and Does Not Demonstrate

The experiment demonstrates that the same hierarchical predictive-coding structure can be used with different probabilistic forms of prediction error.

In particular, it demonstrates:

1. Gaussian prediction errors lead to precision-weighted quadratic error signals.
2. Laplace prediction errors lead to absolute-error costs and bounded error gradients.
3. The choice of error distribution changes the inference and learning signals even when the hierarchical architecture remains unchanged, visibly so in the learned-scale dynamics (Figure 2).
4. Distributional prediction requires a discrepancy appropriate to distributions, such as KL divergence.
5. For categorical prediction, $p - q$ provides a vector-valued prediction-error signal analogous to the scalar error $x - \hat{x}$, and the resulting accuracy gap (Table 3, Figure 4) is large and consistent across seeds.

However, the experiment does **not** constitute a reproduction of the complete experiments of either Rao and Ballard (1999) or Pinchetti et al. (2022).

It is a controlled educational and research prototype designed to make the mathematical distinction visible.

17 Important Limitations

1. **Part A is not the full distribution-to-distribution formulation.**

Part A generalizes the likelihood of a scalar residual. Part B is included specifically to demonstrate distribution-to-distribution prediction.

2. **Only the observation-level error distribution is changed in Part A.**

The higher-level error ϵ_1 remains Gaussian. Therefore the experiment should be described as a controlled observation-level comparison rather than a completely generalized hierarchy.

3. **The synthetic dataset is deliberately simple.**

It uses

$$z_2^* \sim \mathcal{N}(0, 1), \quad z_1^* = \sin(z_2^*),$$

which makes the latent structure easy to understand and evaluate.

4. **Five random seeds are used, but this is not a statistically powered benchmark.**

Results are summarized using mean and standard deviation across five seeds; several of the resulting standard deviations are wide relative to the mean (see Table 1), which should be stated plainly rather than obscured.

5. **The held-out test set comes from the same generated dataset.**

This provides a basic train/test separation but is not equivalent to generating a completely independent test population.

6. **Gradient and state clipping are used.**

Gradients are clipped to a fixed range and latent variables and parameters are bounded for numerical stability. Therefore the observed dynamics include these implementation safeguards.

7. **The Student- t distribution is implemented but not used in the main Part-A comparison.**

It could be used in a future experiment to provide another heavy-tailed error model.

8. **The experiment is not intended to establish that generalized PC is universally better than Gaussian PC.**

The purpose is to demonstrate how changing the probabilistic assumption changes the prediction-error function and therefore the resulting inference and learning dynamics.

18 Conceptual Summary for Presentation

The complete story can be summarized in five steps.

1. Start with the hierarchy

$$z_2 \rightarrow z_1 \rightarrow x$$

Higher levels predict lower levels.

2. Define the predictions

$$\hat{z}_1 = W_2 \sin(z_2)$$

$$\hat{x} = W_1 z_1$$

3. Define the prediction errors

$$\epsilon_1 = z_1 - \hat{z}_1$$

$$\epsilon_0 = x - \hat{x}$$

4. Change the probabilistic model

Gaussian:

$$E_G(\epsilon_0) = \frac{1}{2} \Pi \epsilon_0^2 + \text{normalization}$$

Laplace:

$$E_L(\epsilon_0) = \frac{|\epsilon_0|}{b} + \text{normalization}$$

Therefore,

$$g_G(\epsilon_0) = \Pi \epsilon_0$$

while

$$g_L(\epsilon_0) = \frac{\text{sign}(\epsilon_0)}{b}.$$

5. Generalize from scalar values to distributions

Scalar:

$$x - \hat{x}$$

Distributional:

$$p - q$$

with

$$E = D_{\text{KL}}(q||p).$$

The central message is therefore:

The predictive-coding hierarchy remains the same; what is generalized is the probabilistic representation of prediction and prediction error.

19 Conclusion

The comparison between Gaussian predictive coding and generalized predictive coding can be understood without changing the basic hierarchical mechanism.

The common architecture is

$$z_2 \rightarrow z_1 \rightarrow x,$$

with top-down predictions and bottom-up prediction errors.

For scalar observations, Gaussian predictive coding gives

$$E_G(\epsilon) = -\log p_G(\epsilon) \propto \epsilon^2,$$

which produces the familiar error signal

$$g_G(\epsilon) = \Pi\epsilon.$$

Generalizing the scalar error distribution changes the negative log-likelihood. For the Laplace distribution,

$$E_L(\epsilon) = \frac{|\epsilon|}{b} + \log(2b),$$

with

$$g_L(\epsilon) = \frac{\text{sign}(\epsilon)}{b}.$$

Thus the Gaussian and Laplace models have the same predictive-coding architecture but different mappings from prediction error to update signal, and this is visible empirically both in final test-set metrics (Table 1) and in the stability of the learned scale parameter over training (Figure 2).

The second experiment makes the generalization more explicit. Instead of predicting a scalar value, the model predicts a categorical distribution:

$$p = \text{softmax}(Az_1 + b),$$

against a target distribution q . The prediction error is measured by

$$D_{\text{KL}}(q||p),$$

whose gradient with respect to the logits is

$$p - q.$$

This provides the conceptual bridge from ordinary scalar predictive coding to distributional predictive coding, and produces the largest and most consistent empirical effect in the entire study: an accuracy gap of roughly 0.5–0.6 between the distributional model and a naive scalar baseline, stable across five random seeds (Table 3).

The final interpretation is therefore not that Gaussian predictive coding is simply “wrong” and generalized predictive coding is simply “better.” Rather,

Gaussian PC is one probabilistic choice within a broader PC framework.

The Gaussian assumption produces a quadratic error geometry. Other probability distributions produce different error geometries, and distributional prediction requires discrepancy measures appropriate to the predicted distributions.

The experimental implementation makes this progression explicit:

Gaussian scalar error $\rightarrow$ Laplace scalar error $\rightarrow$ categorical distributional error.

This is the central concept to carry into the presentation.

References

- [1] R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.
- [2] L. Pinchetti, T. Salvatori, Y. Yordanov, B. Millidge, Y. Song, and T. Lukasiewicz. Predictive Coding beyond Gaussian Distributions. In *Advances in Neural Information Processing Systems* 35, pages 1280–1293, 2022.

- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [4] J. C. R. Whittington and R. Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5):1229–1262, 2017.
- [5] K. Friston. A theory of cortical responses. *Philosophical Transactions of the Royal Society B*, 360(1456):815–836, 2005.

A Notation Used in the Current Implementation

Symbol	Meaning
z_2^*	True higher-level latent used to generate the data
z_1^*	True intermediate latent, $\sin(z_2^*)$
x	Observed noisy input
z_2	Model’s inferred higher-level latent
z_1	Model’s inferred intermediate latent
$\hat{z}_1$	Top-down prediction of z_1
$\hat{x}$	Top-down prediction of x
ϵ_1	Intermediate prediction error, $z_1 - \hat{z}_1$
ϵ_0	Observation prediction error, $x - \hat{x}$
W_2	Parameter generating $\hat{z}_1$
W_1	Parameter generating $\hat{x}$
Π	Gaussian precision
b	Laplace scale parameter
q	Target categorical distribution
p	Predicted categorical distribution
A, b	Parameters of the categorical prediction layer

Table 4: Notation used throughout the current experiment.